Test report 9 – desktop app, file watcher, Git, section view, CLI↔UI

cad-app (Tauri 2), cad-desktop, cad-ipc, cad-cli, web · 2026-10-23 · Branch worktree-agent-ac30a6a8716573971 · Commit 3415336 · rustc 1.97.0 (2d8144b78 2026-07-07) · WebKitGTK 2.52.6 · 4 cores (Intel(R) Xeon(R) Processor @ 2.10GHz) · Xvfb without GPU

Tauri-E2E (tauri-driver, Xvfb)	Playwright (Chromium)	cargo test --workspace	clippy · fmt · actionlint	Start to scene	Keystroke → scene (median)
11 / 11	**77 / 77**	**338 / 338**	**clean · clean · clean**	**1,719 ms**	**74 / 34 ms**
real app, real kernel	of which 7 section, 70 existing	0 ignored		session → first complete scene	native (Tauri) / Wasm (Chromium)

Kernel-heavy change	Moving the section
551 / 1,273 ms	**12.2 fps**
native / Wasm (median)	Chromium SwiftShader; Tauri 37.6

1 Scope and architecture

The desktop version runs the same React app as the browser; new are the Tauri app (`crates/cad-app`), the backend without Tauri (`crates/cad-desktop`: engine process with supervisor, binary format, files, watcher, Git), the local IPC (`crates/cad-ipc`) and, in the web app, `TauriTransport`, the desktop platform and the section view (browser and desktop).

```
cad datei.scad  (app process, feature gui)
  Main thread: Tauri event loop + webview (React app, TauriTransport instead of worker)
  Commands: engine_post · open/save/save_as · watch_files · git_hunks · ui_reply · …
  Channel (Tauri 2, raw): binary batches  ◄──────────── stdout ── cad engine-host (engine process)
  Supervisor (cad-desktop::engine) ──── stdin: post [ToEngine…] ─►  engine thread 256 MiB stack:
    Watchdog 15 s: kill process + restart, BlockKeys                Session::post / step(16 ms)
  cad-watch: notify → file-changed / file-removed                   SMLib native (FFI)
  cad-ui-ipc: Unix socket ui-<pid> (cad-ipc) ◄── cad ui status | show | screenshot
```

- **Engine:** native `cad-session` with natively linked SMLib in a separate process (`cad engine-host`, the same binary), containing an engine thread with a 256 MiB stack and the same loop as the worker (post messages, a `SetSource` followed by another one is dropped, then `step(16 ms)`, new messages in between). A process rather than just a thread, because SMLib and its C facade have global state (handle table, messages): a hanging call can only be ended together with the process; a replacement thread next to it would share the state. The watchdog in the app process counts sent and accepted messages (`seen`) and replaces the engine after 15 s without a response, with the same `BlockKeys`/`SetFiles`/`SetContext`/`SetSource` as `WorkerTransport`.
- **Transport:** `EngineTransport` stays; `TauriTransport` bundles UI messages per microtask into one `engine_post` command (order is kept) and receives batches over a Tauri 2 channel as `ArrayBuffer`: `[u32 Kopflänge][u32 Datenlänge][Kopf-JSON][Daten]` (header length, data length, header JSON, data), mesh arrays as `{"$b": [offset, länge, typ]}`, typed-array views without copying in the webview. The desktop UI is built without worker and Wasm (`pnpm build:tauri`) and embedded in the binary.
- **One binary:** `cad` with feature `gui` (default). `cad`, `cad datei.scad`, `cad ordner/` start the UI; without `DISPLAY`/`WAYLAND_DISPLAY` or without `gui`: message listing the headless commands, exit 2. All subcommands remain.
- **IPC (`cad-ipc`, reusable):** endpoints as Unix sockets in `$CAD_IPC_DIR`, else `$XDG_RUNTIME_DIR/controller-cad`, else `$TMPDIR/controller-cad-<uid>` (folder 0700, owner checked, sockets 0600, peer UID checked); frames with version field `v`, kinds request/response/event, binary part; stale sockets (connection refused) are removed on

bind, connect and list. API: `Server::bind(name)`, `Server::serve(handler)`, `connect(name)`, `Conn::request/request_with/respond/send_event/next_event`, `list(prefix)`.

2 Decisions

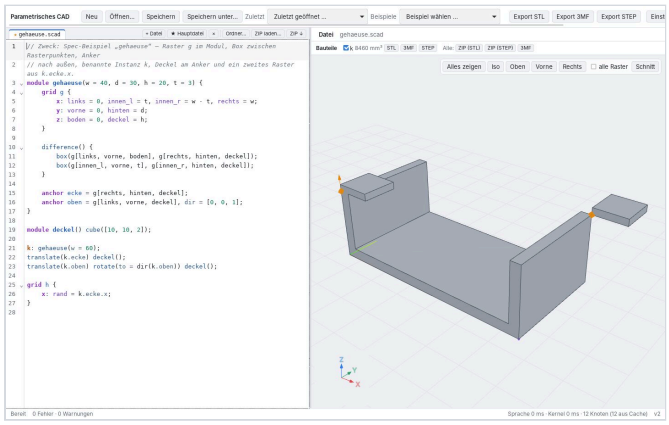
Topic	Decision	Reason
Git integration	`git` command for the HEAD version (`rev-parse`, `show HEAD:pfad`), diff in-process with `similar`	One blob per file and HEAD state (cached, HEAD re-checked at most every 5 s); typing does not start a process. `git2` would need libgit2 (C build), `gix` would add well over a hundred crates to every binary; `git` is present wherever there are repositories. Untracked files or files not in HEAD: no markers.
Feature `gui`	on by default	The workspace contains `cad-app` anyway (`cargo test --workspace` builds it); the only extra cost is the WebKitGTK packages on Linux (CI: `scripts/ci/desktop.sh deps`). Headless: `--no-default-features`. Without `web/dist-tauri` the build embeds a placeholder page.
Engine	separate process, containing a thread with a large stack	see section 1 (global SMLib state, watchdog)
Auto-save	off by default, delay configurable (1000 ms)	Whoever works on the file at the same time as an agent should decide when their state reaches the disk; the notification bar protects both sides.
Watcher	`notify` on the folders (not the files), content comparison by hash	atomic saving (editors, `cad set`) replaces the file; the app's own saves register the new content beforehand and never come back as "external".
Reload	as minimal line changes, separate undo step (`isolateHistory`)	Cursor, selection and scroll position are kept; Ctrl+Z restores the user's own state.

3 Desktop app (Tauri E2E with tauri-driver and WebKitWebDriver under Xvfb)

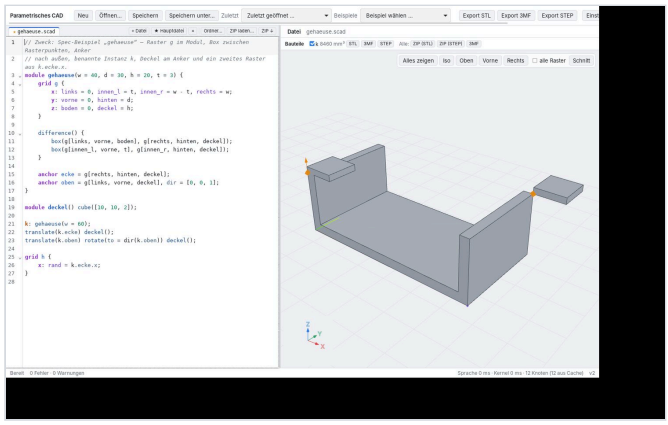
Binary `target/release/cad`, 11/11 passed. The run starts the real app with a file via WebDriver, changes files from the shell, uses the real CLI (`cad check`, `cad set`, `cad ui …`) and a temporary Git repository. tauri-driver ran stably under Xvfb; images: WebDriver screenshots of the webview and an X screenshot of the window.

T01 App starts with a file, scene appears (native kernel) passed 2,060 ms

- ✓ Tauri platform detected (TauriTransport, no worker)
- ✓ file from the command line opened
- ✓ no diagnostics
- ✓ project folder /tmp/cad-desktop-e2e-BcTy3G/repo
- ✓ meshes binary over the channel (102144 bytes)



Start with gehaeuse.scad



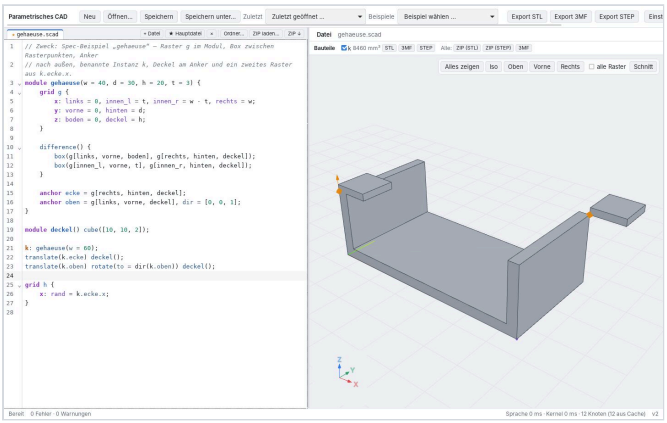
Window under Xvfb (X screenshot)

T02 External change without unsaved changes: silently reloaded, undo as a separate step passed

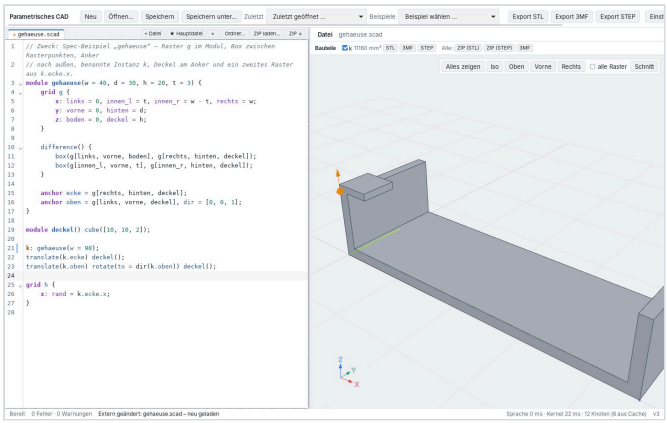
618 ms

The file is changed externally (`k: gehaeuse(w = 60)` → `w = 90`); the editor has no unsaved changes.

- ✓ test change applies
- ✓ editor shows the new content
- ✓ view evaluated
- ✓ cursor stays in line 24
- ✓ not marked as unsaved
- ✓ model wider (bounding box x 0...100)
- ✓ undo restores the old state (separate step)
- ✓ redo



before (w = 60)

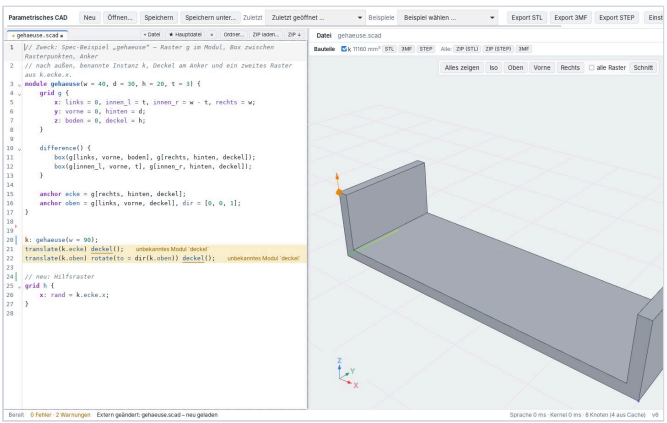


after the external change (w = 90), reloaded without user action

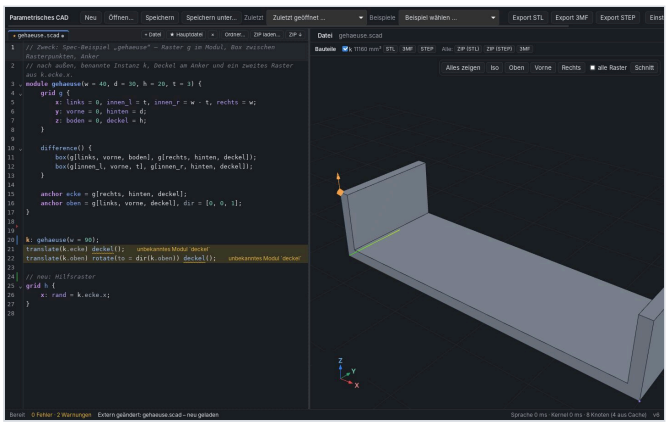
T03 Git markers in the gutter (new, changed, deleted) passed 846 ms

Buffer against HEAD: line 21 changed (blue), one new comment line (green), the module `deckel` deleted (red triangle between the lines). The markers are 3 px wide with opacity 0.5–0.55 (tuned for light/dark); the title (hover) shows the old lines.

- ✓ changed line 21 (w = 90) marked blue
- ✓ new line green
- ✓ deleted line as triangle
- ✓ 3 markers in the DOM
- ✓ narrow bar (≤ 5 px)
- ✓ low opacity
- ✓ hover shows old lines



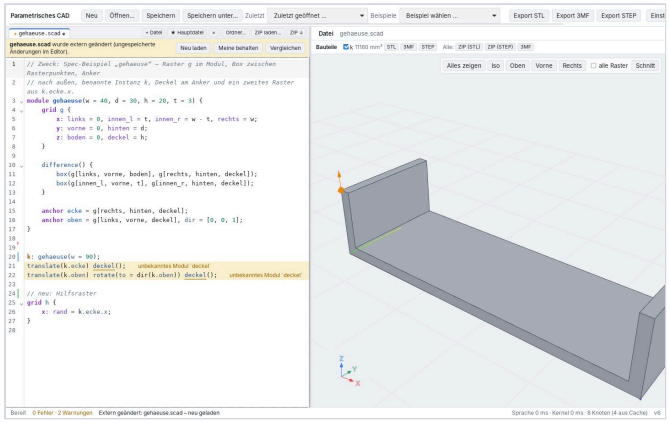
Git markers: changed (blue), new (green), deleted (red triangle)



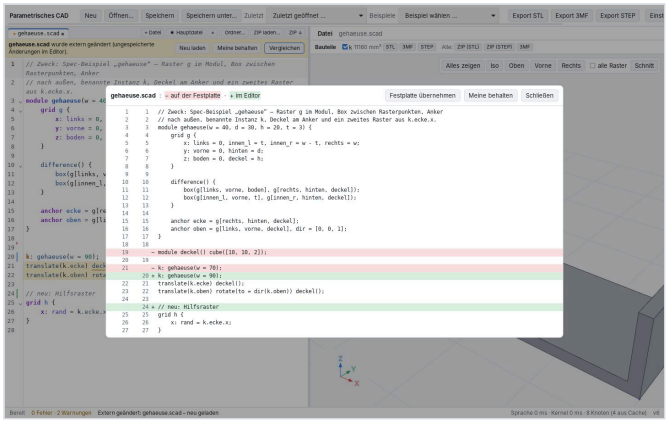
the same in the dark theme

T04 External change with unsaved changes: notification bar, Compare, Keep mine, save without loop passed 2,358 ms

- ✓ buffer is unsaved
- ✓ editor keeps its own state
- ✓ comparison shows + and – lines
- ✓ bar gone after "Keep mine" (German UI: „Meine behalten“)
- ✓ no reload loop (message: „Gespeichert: gehaeuse.scad“)
- ✓ no notification bar after saving
- ✓ saved, no longer marked
- ✓ "Reload" (German UI: „Neu laden“) takes the disk version



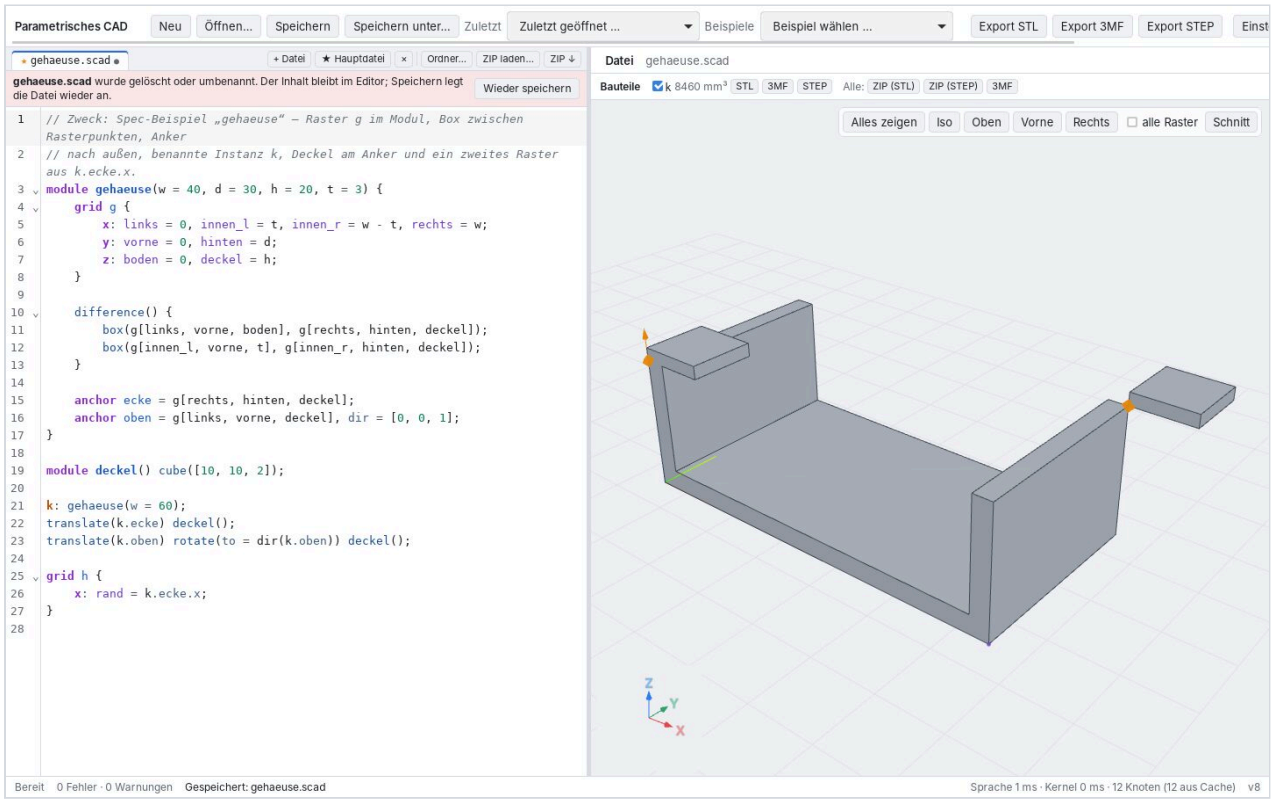
"File was changed externally" (German UI: „Datei wurde extern geändert“) with Reload / Keep mine / Compare



Compare: disk (–) against editor (+)

T05 Deleted and renamed file passed 1,235 ms

- ✓ saving recreates the file
- ✓ no bar any more



File renamed/deleted: content stays, "Save again" (German UI: „Wieder speichern“)

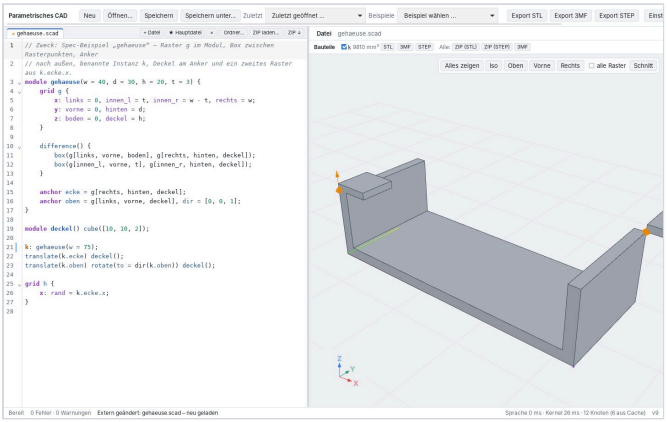
4 Workflow "agent changes file → UI updates"

The user has the file open in the UI; an agent changes it from the shell and checks it with the CLI. Each image is the UI after the step, without any action in the UI. Time from writing to the drawn scene: 232 ms, 249 ms, 188 ms (incl. `cad check`/`cad set`).

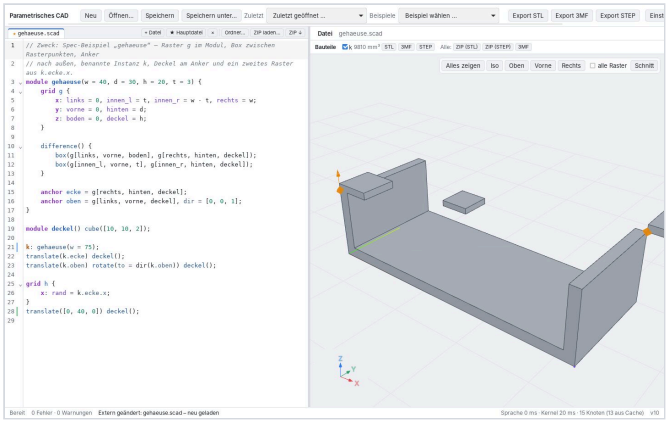
T06 Agent workflow: change file from the shell, `cad check`, UI follows (image sequence) passed

983 ms

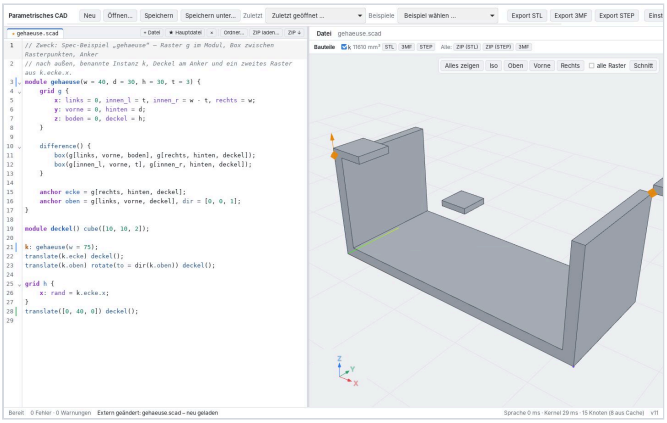
- ✓ `cad check --json`: exit 0, 0 errors
- ✓ editor = file after step 1
- ✓ `cad check --json`: exit 0, 0 errors
- ✓ editor = file after step 2
- ✓ `cad set --param gehaeuse.h=30`: exit 0
- ✓ editor = file after step 3



1. Agent changes w = 60 → 75 and checks with `cad check`



2. Agent adds a second lid

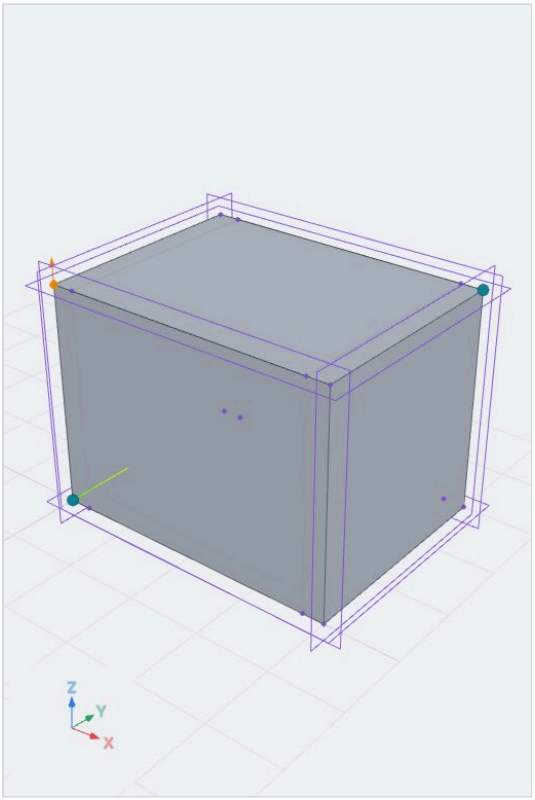


3. Agent sets h via `cad set --param gehaeuse.h=30`

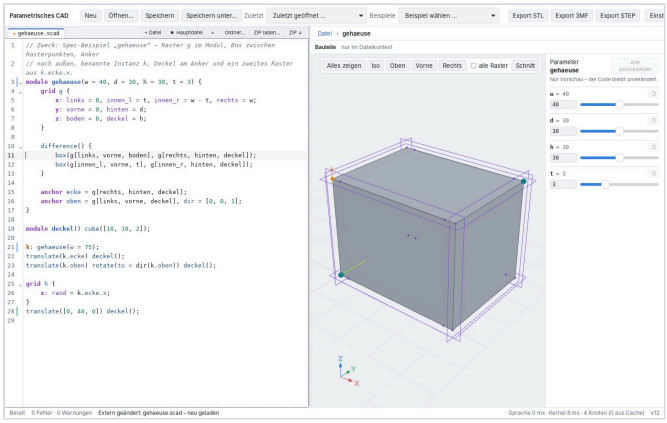
T07 `cad ui status` / `show` / `screenshot` against the running app passed 895 ms

Coupling CLI ↔ running UI via `cad-ipc`.

- ✓ status exit 0
- ✓ open file and folder
- ✓ cursor line
- ✓ show exit 0
- ✓ cursor in line 11
- ✓ status sees line 11
- ✓ context module gehaeuse (gehaeuse)
- ✓ selected node {"idx":2,"label":"box","line":11}
- ✓ values of the parameter bar
- ✓ diagnostics
- ✓ screenshot exit 0
- ✓ PNG 33173 bytes
- ✓ show --module deckel: context deckel
- ✓ unknown module: exit 1 („Modul gibtsnicht nicht gefunden“)



`cad ui screenshot`: the 3D view as the user sees it



after `cad ui show gehaeuse.scad --line 11`

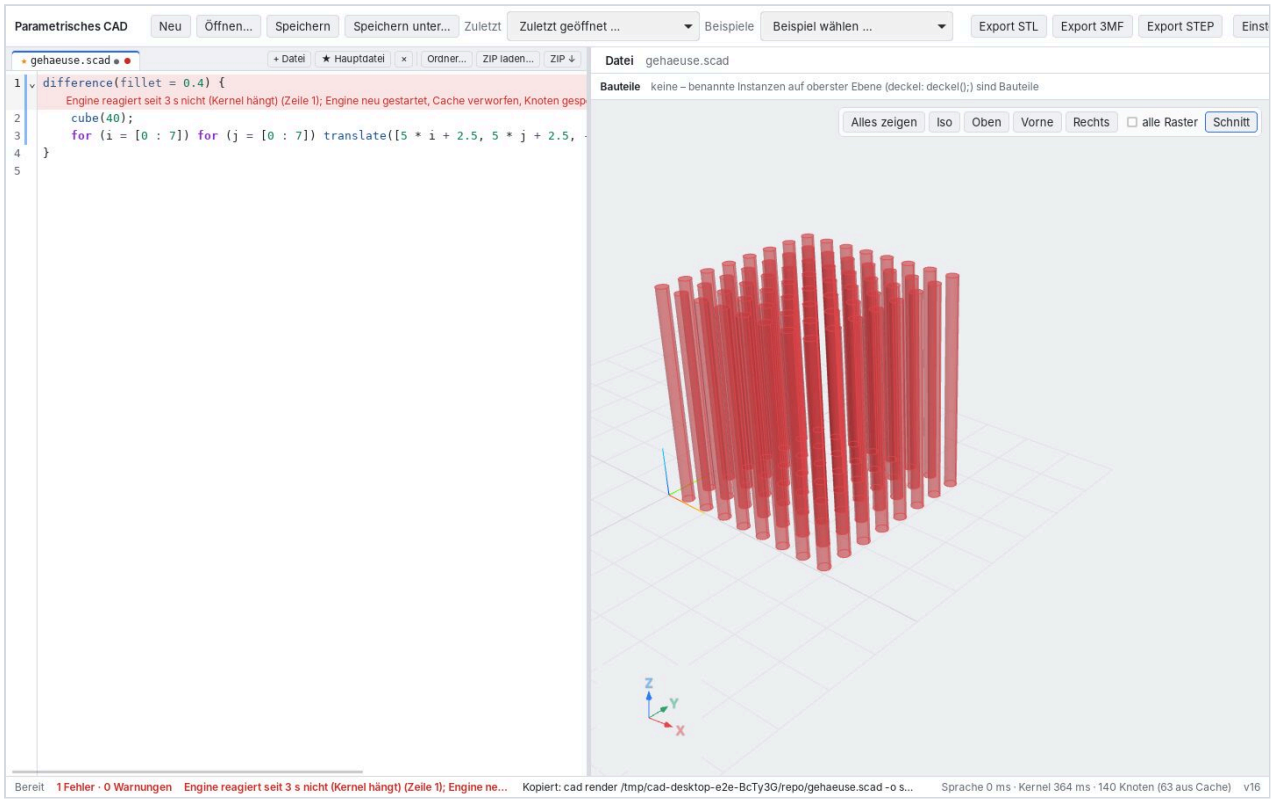
Output of `cad ui status`:

```
UI pid 11196 (ui-11196)
  Projekt /tmp/…/repo (Hauptdatei gehaeuse.scad)
  Dateien: gehaeuse.scad
  Cursor: gehaeuse.scad Zeile 11 Spalte 1
  Auswahl: [2] box
  Kontext: Modul gehaeuse
  Parameterleiste: d = 30, h = 30, t = 3, w = 40
  Diagnosen: 0 Fehler, 0 Warnungen
  Szene v12 (vollständig), 4 Knoten
```

T09 Watchdog in Tauri: hanging kernel → engine process replaced, node blocked passed 7,822 ms

Hanging kernel (test message `__debugHang` in the engine process, watchdog 3 s for the test): (a) when idle, (b) during a slow kernel job (8×8 holes with fillet, native ~3.6 s).

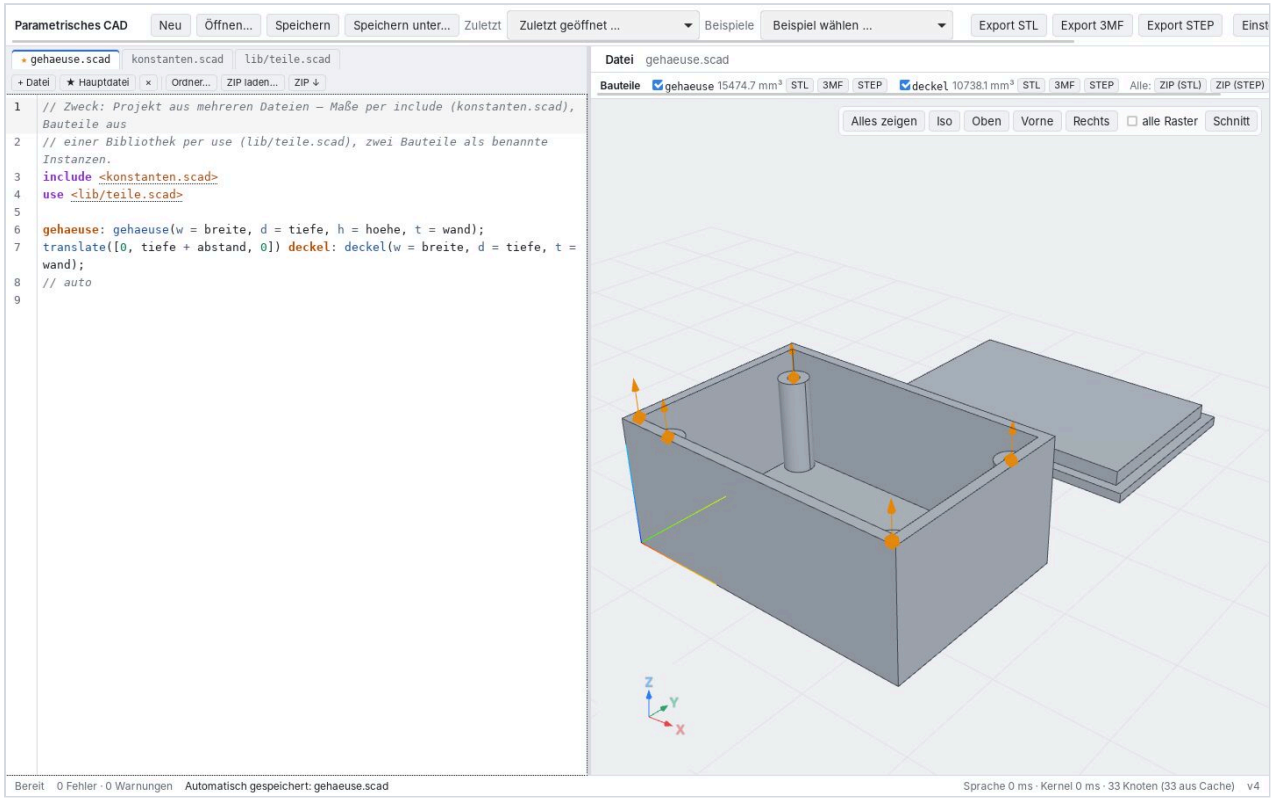
- ✓ "Engine not responding for 3 s (kernel hangs); engine restarted, cache discarded"
- ✓ next change is evaluated normally
- ✓ "Engine not responding for 3 s (kernel hangs)" (line 1); "engine restarted, cache discarded, node blocked"
- ✓ 1 blocked key
- ✓ blocked node reports an error (1)
- ✓ scene after the restart



Watchdog: engine process replaced, hanging node blocked (line red)

T10 Folder as project, auto-save passed 2,786 ms

- ✓ main file gehaeuse.scad
- ✓ 3 files (gehaeuse.scad, konstanten.scad, lib/teile.scad)
- ✓ included file reloaded
- ✓ clean after auto-save



Folder as project (3 files), auto-save on

T11 Keystroke → scene, native (measurement) passed 7,969 ms

- ✓ 8 measurements

5 3D section view

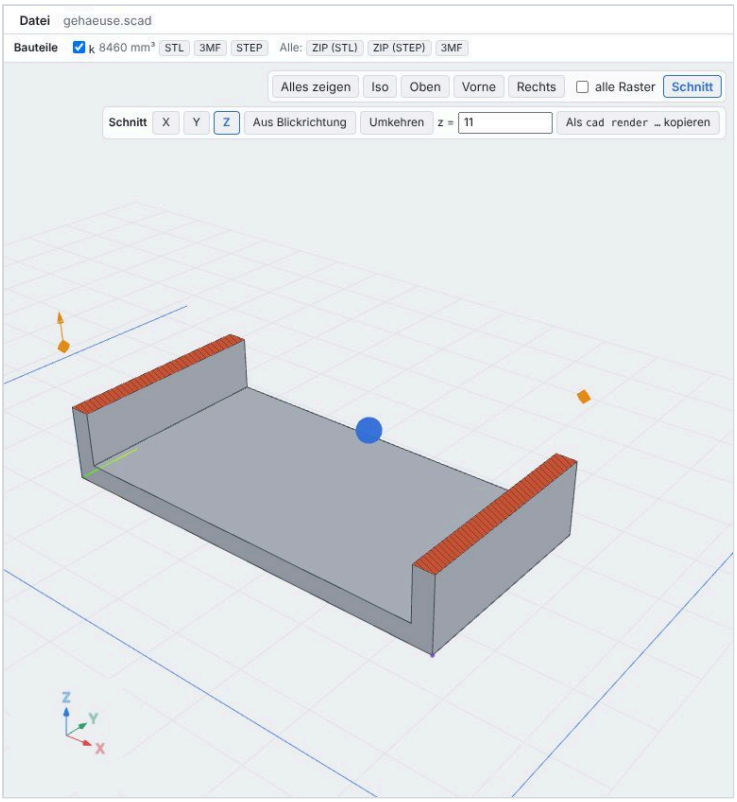
Tool "Section" (German UI: „Schnitt“) in the view (browser and Tauri): plane X/Y/Z through the model center or free (normal from the view direction or entered), handle (blue dot) for dragging along the normal with snapping to grid planes of the context module and to anchors (Alt: free, 0.1 mm), flip, filled cap and contour. Method as in the CLI renderer: clipping plane on all model materials, per opaque body one parity pass (stencil `INVERT` over all fragments of the clipped mesh) and a large quad in the plane that draws the cap where the bit is set (depth-tested, hatched, direction alternating per body). Convention exactly as `cad render --section`: plane through p with normal n, the side n points into is removed; "Copy as `cad render …`" writes `--section a=v [--section-flip]` or `--section-plane p,n` with `--module` and the camera (`--azimuth`/`--elevation`).

X01 Section z = 11 with cap passed

"Section" in the view: plane z through the model center (z = 11), z > 11 is removed. The section face is filled and hatched (stencil cap, color as in the CLI renderer), the contour as a line; status line „Schnitt z = 11 · entfernt z > 11“ ("Section z = 11 · removed z > 11").

✓ cap pixels before 27, after 6452

✓ 1 body with cap, 18 contour segments

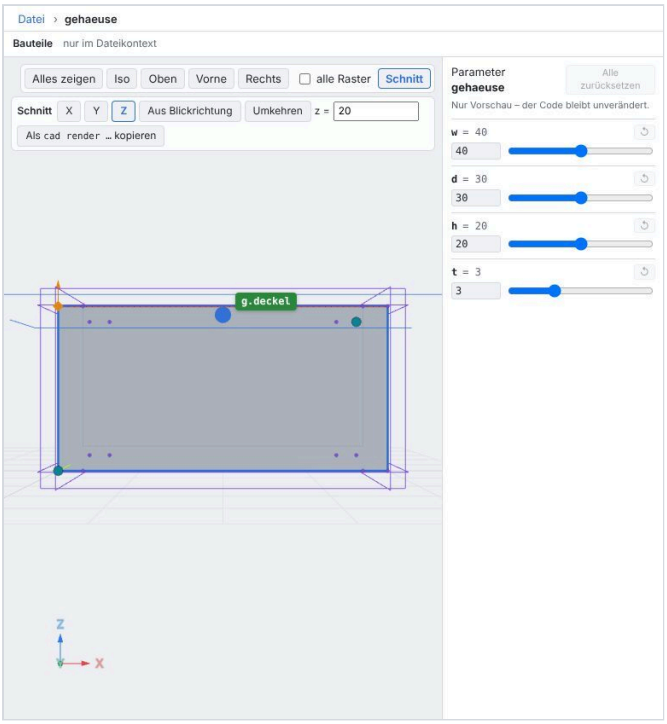


Section z = 11 (iso)

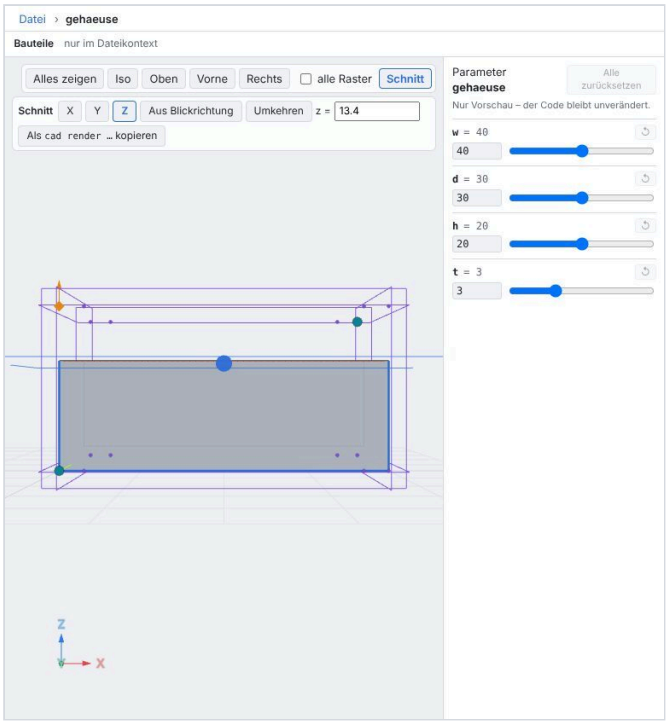
X02 Drag and snap the plane passed

In module context gehaeuse (grid g): handle (blue dot) dragged upwards, the plane snaps to g.deckel (z = 20, green label), downwards to g.boden; with Alt free (rounded to 0.1 mm).

✓ snapping: g.deckel (z = 20), g.boden (z = 0)
✓ free: z = 13.4
✓ fps while dragging: 7, 25, 24



while dragging: snaps to g.deckel

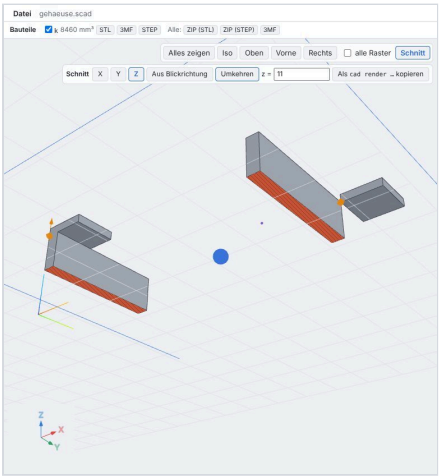


dragged freely with Alt

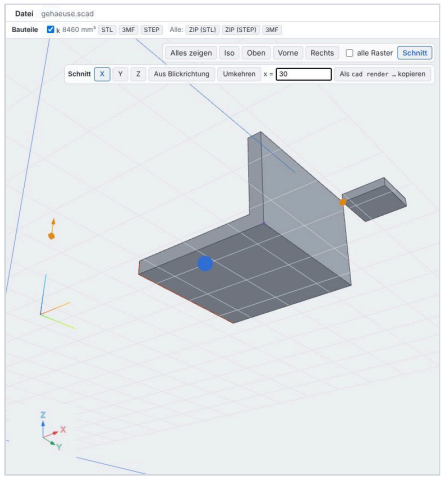
X03 Flip, axes, free plane passed

"Flip" (German UI: „Umkehren“) keeps the upper half (n = −z, corresponds to --section-flip); X with value 30; "From view direction" (German UI: „Aus Blickrichtung“) places a free plane perpendicular to the view direction, the normal can be entered (1, 1, 1).

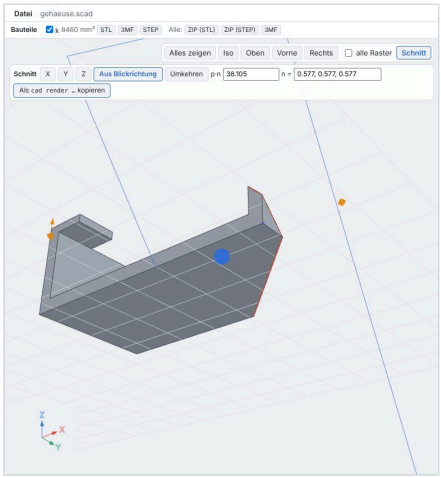
✓ CLI: cad render gehaeuse.scad -o schnitt.png --section-plane 35,20,11,0.5774,0.5774,0.5774 --azimuth 38.7 --elevation -29.3



z flipped (removes the lower half), from below



x = 30



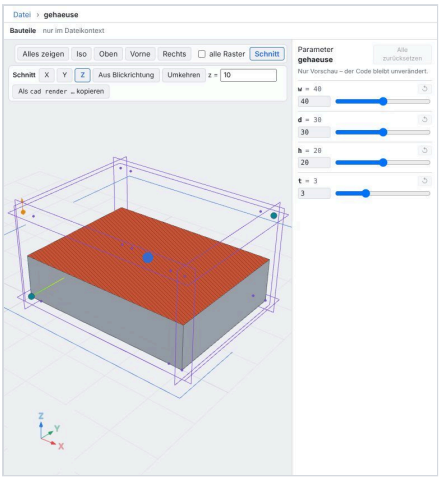
free, n = (1, 1, 1)/√3

X04 Section combined with highlighting, zones, transparency passed

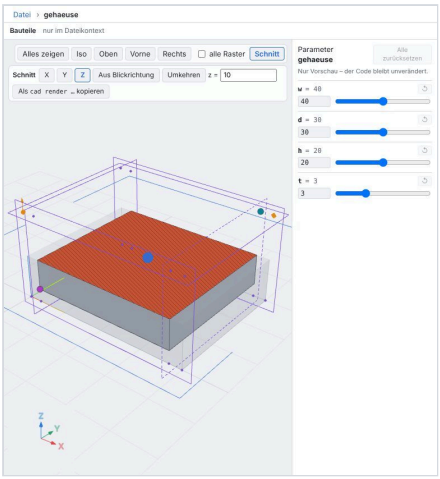
Cursor on a line: its bodies in full and with cap, the rest dimmed (only clipped, without cap); subtracted body red transparent without cap; zone (blue) in the fillet example; handles stay above the section.

✓ caps = full bodies (1)

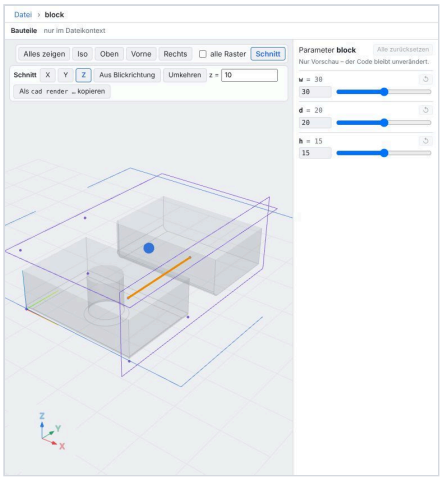
✓ 0 handles drawn



Line 11 highlighted



Line 12: subtracted body

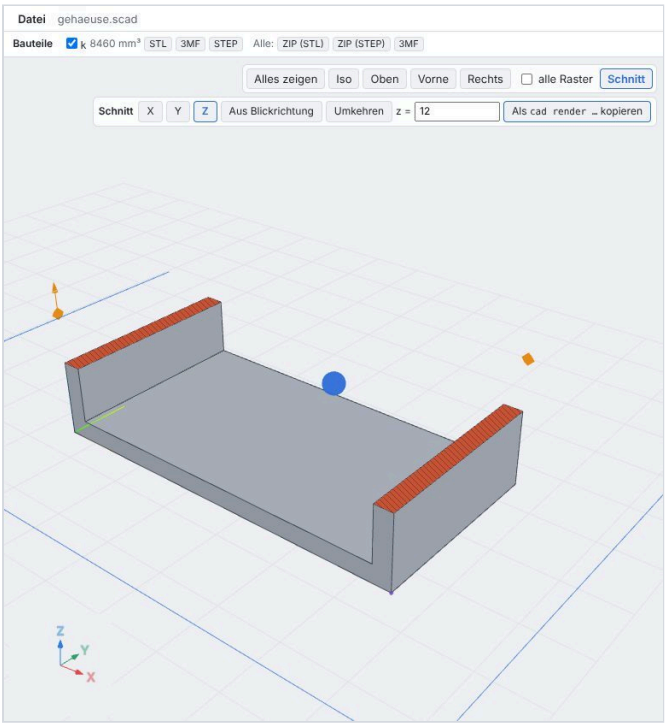


Zone (line 17)

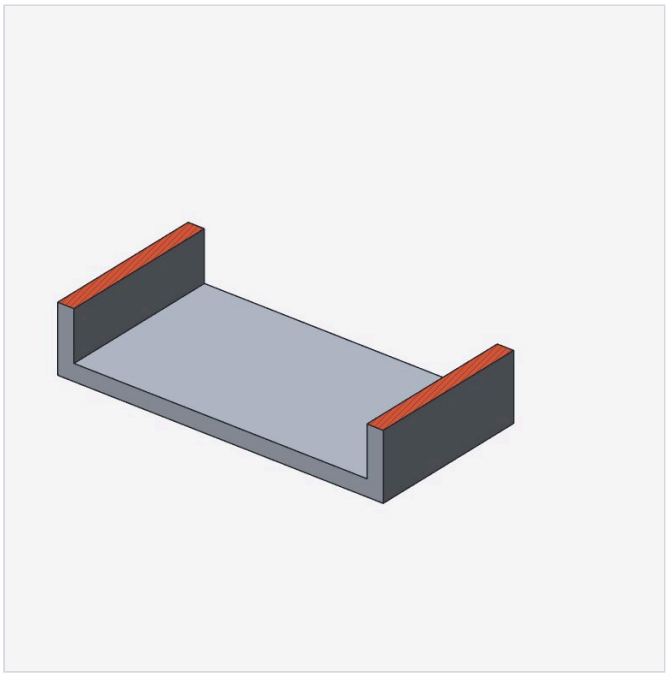
X05 Same section in UI and CLI passed

"Copy as cad render ..." gives `cad render gehaeuse.scad -o schnitt.png --section z=12 --azimuth 38.7 --elevation 29.3`; run in the CLI, the image shows the same section (cap share of drawn pixels UI 6.7 %, CLI 6.2 %, deviation 7 %; perspective in the UI, orthographic in the CLI).

✓ cap pixels UI 6429 / 96597, CLI 5335 / 85769



UI (WebGL)



CLI (software renderer), copied call

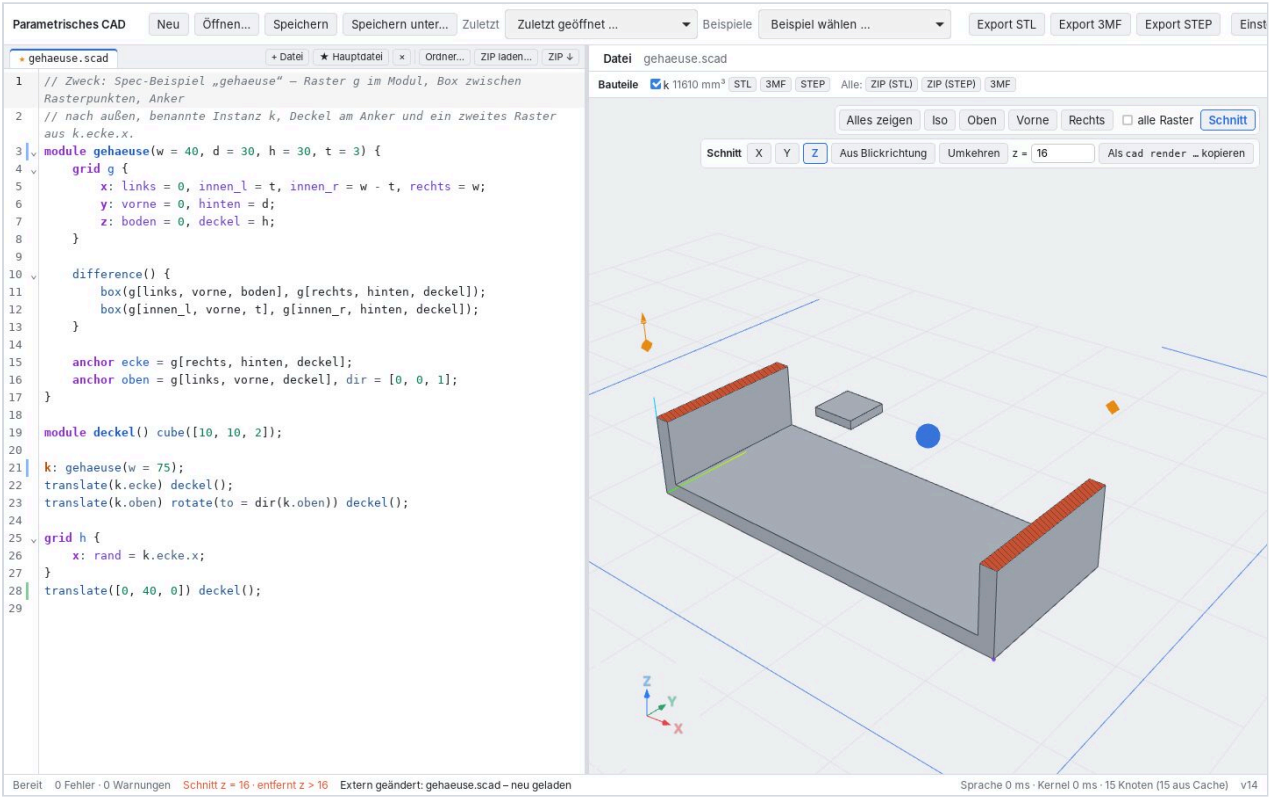
X06 Frame rate while dragging the section passed

Plane moved for 120 frames (each frame a new position, cap and contour rebuilt): 12.2 fps in Chromium with SwiftShader (software GL, no GPU); rotating with section 14.7 fps, without section 21.8 fps – the section costs little more than the drawing itself.

✓ 120 frames in 9809 ms

T08 Section view in Tauri (cap, contour, dragging, CLI call) passed 3,603 ms

✓ 1 body with cap
✓ 18 contour segments
✓ cap visible (4022 red pixels)
✓ moving the plane: 37.6 fps (WebKitGTK, software GL)
✓ cad render /tmp/cad-desktop-e2e-BcTy3G/repo/gehaeuse.scad -o schnitt.png --section z=16 --azimuth 38.7 --elevation 28.7



Section z in Tauri: filled, hatched cap and contour

Copied call in Tauri: `cad render /tmp/…/repo/gehaeuse.scad -o schnitt.png --section z=16 --azimuth 38.7 --elevation 28.7`

6 Measurements

Measurement	Value	Remark
App start to first complete scene	1,719 ms	create WebDriver session (process start, webview, engine process, SMLib) until the scene is drawn; of which in the webview until the first scene 861 ms
Keystroke → scene, native (Tauri)	median 74 ms (79, 60, 76, 68, 90, 85, 71, 56)	Backspace/digit in `w = 60` (gehaeuse), real keystrokes via WebDriver; time from the editor change until the scene is drawn (incl. 8 ms debounce, WebKitGTK with software GL)

Keystroke → scene, Wasm (Chromium)	median 34 ms (73, 39, 33, 19, 32, 36, 33, 36)	the same change in the browser (worker, smlib.wasm), headless Chromium with SwiftShader
of which engine (SetSource → complete scene)	native median 13 ms, Wasm median 5 ms	evaluation and kernel without drawing, same keystrokes
Kernel-heavy change (16 holes with fillet, radius 2 → 2.5 → 3)	native 576 / 532 / 551 ms, Wasm 1,273 / 929 / 1,377 ms	SetSource → complete scene; first value incl. empty cache for the new model
External change → scene	178 ms	write file until new scene drawn (debounce 50 ms in the watcher)
Moving the section plane (Chromium)	12.2 fps	fillet_zonen, 120 frames, each position new (cap, contour); rotating without section 21.8 fps, with section 14.7 fps – software GL without GPU, the 60 fps target applies to GPUs
Moving the section plane (Tauri)	37.6 fps	gehaeuse, WebKitGTK under Xvfb (llvmpipe)
Dragging the handle (Chromium)	7 fps, 25 fps, 24 fps	X02: three real mouse drags (~12 moves each), frame rate of the view while dragging
Cap share UI ↔ CLI	6.7 % / 6.2 %	share of cap pixels in the drawn pixels, deviation 7 % (limit 35 %)

For small changes (keystroke in *gehaeuse*, kernel almost entirely from the cache) the desktop app is not faster: the engine computes for only a few milliseconds; the time is dominated by debounce, the path via Tauri command, engine process and channel (native 13 ms against 5 ms in the worker) and the software rendering of WebKitGTK under Xvfb. As soon as the kernel computes, the native kernel wins clearly (row "Kernel-heavy change"). All measurements without GPU; meaningful only relative to each other.

7 Tests

- `cargo test --workspace`: 338 passed, 0 failed, 0 ignored; `cargo clippy --workspace --all-targets -- -D warnings`: no findings; `cargo fmt --check`: no findings.
- Web: `pnpm typecheck` no findings; Vitest Tests 186 passed (186) (new: `tauriTransport.test.ts`, `textDiff.test.ts`, `section.test.ts`); Playwright 77/77 (all 70 existing scenarios and 7 new ones for the section view).
- CI: workflow `rust` installs the Tauri packages (`scripts/ci/desktop.sh deps`), new workflow `desktop` (UI for Tauri, release build, Tauri E2E under Xvfb, results as artifact); `actionlint` 1.7.7 over all workflows: no findings.

Rust test (new)	File	Result
tests::base64	unittests src/lib.rs	ok
launch::tests::decisions	unittests src/main.rs	ok
geom::tests::distance_of_cubes	unittests src/main.rs	ok
ui_commands_talk_to_a_running_ui	tests/agent.rs	ok
ui_commands_without_running_ui	tests/agent.rs	ok
ui_entry_without_display	tests/agent.rs	ok
diff_hunks_kinds	tests/backend.rs	ok
open_file_loads_includes	tests/backend.rs	ok
open_folder_chooses_main	tests/backend.rs	ok
atomic_write_and_recent	tests/backend.rs	ok
supervisor_evaluates_and_sends_meshes	tests/backend.rs	ok
head_base_of_tracked_and_untracked_files	tests/backend.rs	ok
wire_batch_has_binary_meshes	tests/backend.rs	ok
supervisor_watchdog_when_idle_engine_hangs	tests/backend.rs	ok

supervisor_watchdog_replaces_engine_and_blocks_key	tests/backend.rs	ok
watcher_reports_external_changes_only	tests/backend.rs	ok
frame_roundtrip_with_data	tests/ipc.rs	ok
runtime_dir_is_private	tests/ipc.rs	ok
stale_socket_is_removed	tests/ipc.rs	ok
version_mismatch_is_reported	tests/ipc.rs	ok
request_response_events_and_stale_cleanup	tests/ipc.rs	ok
serve_handles_requests	tests/ipc.rs	ok

Backend (`cad-desktop/tests/backend.rs`): open file with includes, folder and choice of main file, atomic save and recently opened, watcher (external change, own save ignored, atomic replace, same content, include, unrelated files, deleted/recreated, renamed), Git hunks (kinds, CRLF, end of file) and HEAD base in a temporary repository (untracked, new commit, subfolder), binary format, supervisor (evaluation, meshes, watchdog with block, watchdog when idle). IPC (`cad-ipc/tests/ipc.rs`): frames, version, request/response/event, stale sockets, permissions, multiple clients. CLI (`agent.rs`, `launch.rs`): no display → exit 2 + message, `cad ui …` without UI → exit 3, `cad ui` against a test endpoint.

8 Known limitations

- Control IPC only on Unix (Linux, macOS); on Windows `cad-ipc` reports "not implemented" (named pipes missing), the app itself is untested there. One window per app process; `cad datei.scad` starts another process instead of using the running window (`cad ui show` does that).
- Watchdog tests block the engine process via a test message; a real hanging SMLib call was not reproduced. The block lasts until the app is restarted.
- Watcher: renaming appears as deletion (bar "deleted or renamed"), the new name is not tracked; an external change to exactly the content of the UI goes unnoticed (intended).
- Git only against HEAD (not against the index), via the `git` command; without `git` no markers.
- Native dialogs (Open, Folder, Save as) are not automated in the E2E tests; tested are opening via the command line, `cad ui show`, recently opened (Rust) and saving.
- Section: one plane; caps only for opaque, closed bodies; contour on the CPU per position; handles, grids and anchors are not clipped. The copied CLI call reproduces section and view direction, not the framing (UI perspective, CLI orthographic).
- Measurements without GPU (Xvfb/llvmpipe, SwiftShader).